Flow So far

Lecture #1 & 2

1. מבנה רכיבי לינוקס וההיררכיה המערכתית
ארכיטקטורת לינוקס מובנית בצורה שכבתית ומחולקת לשני מרחבים (Spaces) עיקריים: מרחב הליבה (Kernel Space) ומרחב המשתמש (User Space).

2. הגנה וארכיטקטורת המעבד: User Mode מול Kernel Mode

3. פסיקות (interrupts) וקריאות מערכת [= system calls]: המעבר המבוקר מ- user space ל- kernel space

Lecture #3: Working in a Linux Environment – the bash

1. partitioning and installation
2. bootup and kernel initialization
3. bash – The End User's Perspective
initial description
more: in lecture #4
more: learn by yourself
4. bash – The Programmer's Perspective
next lectures

Lecture #4 {This Lecture !!} – Linux FS

1. *FS and partitioning – short recap*
2. *FS and installation – short recap*
3. *FS - The End User's Perspective – short recap*
4. *FS - The Application Programmer's Perspective – short recap*
5. *FS - necessary internal details*
   - *inode table + data blocks*
   - *{NVM} inode struct*
   - *in-core inode struct*
6. *FS & programs {processes}*
7. *FS - The System Programmer's Perspective*

The Application Programmer's Perspective

- A file in Linux is a sequence of bytes.
- Different programs expect various levels of structure, <u>but the kernel does not impose a structure on files</u>.

Lecture #4 {This Lecture !!} – Linux FS

Standard C library{} functions:

```
fopen()   /* the FILE data type.
             It is an opaque data type, meaning its internal members are not directly
             accessible to the programmer */
fread()   /* unformatted */
scanf(), fscanf()    /* formatted */

fwrite() /* unformatted */
printf(), fprintf()   /* formatted */

fclose()
```

stdin, stdout & stderr

stdin, stdout & stderr – <u>are files !!!</u>

scanf() = fascanf(0, ...)

Are opened automatically !

When the program (more accurate: the process starts running)

All other files {except stdin, stdout & stderr}

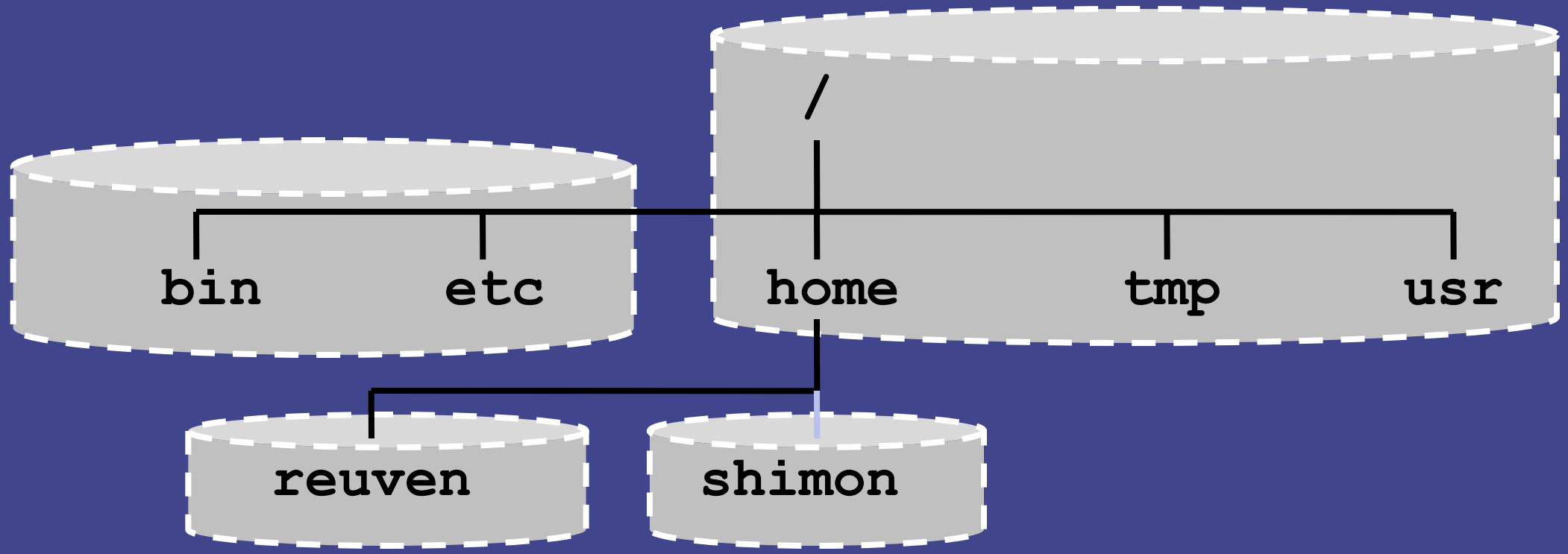
<u>All other files must be opened explicitly by the program</u>
By invoking fopen()

Otherwise, any attempt to access the file, to read andqor write, will end up with a failure !

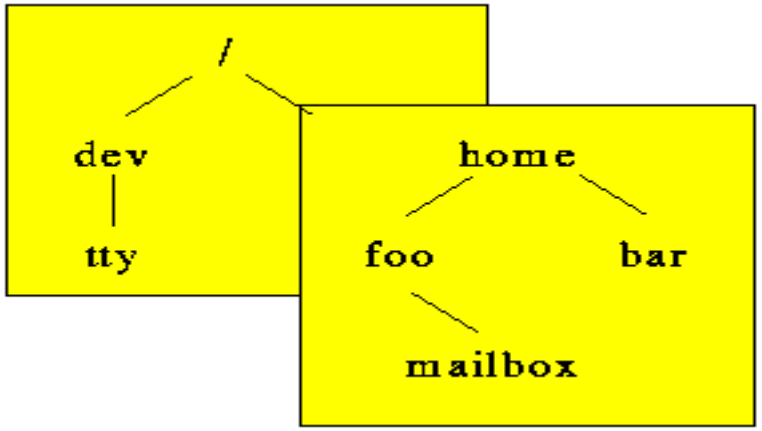
necessary internal details

Disk vs. Filesystem

- The entire hierarchy can actually include many disk drives.
  - some directories can be on other computers

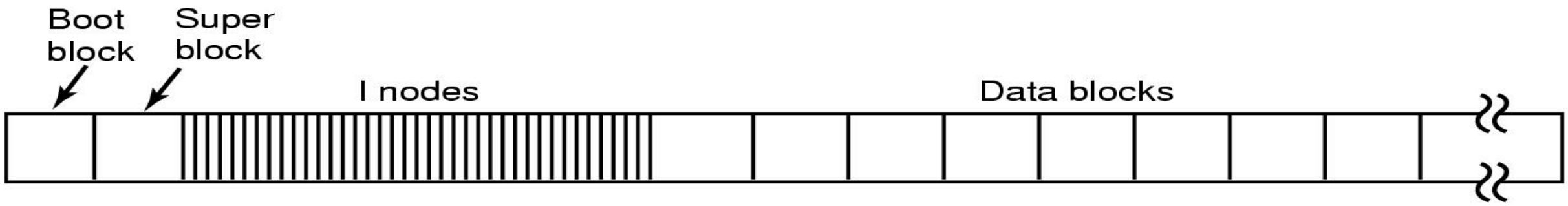


UNIX file system tree built from multiple disk partitions

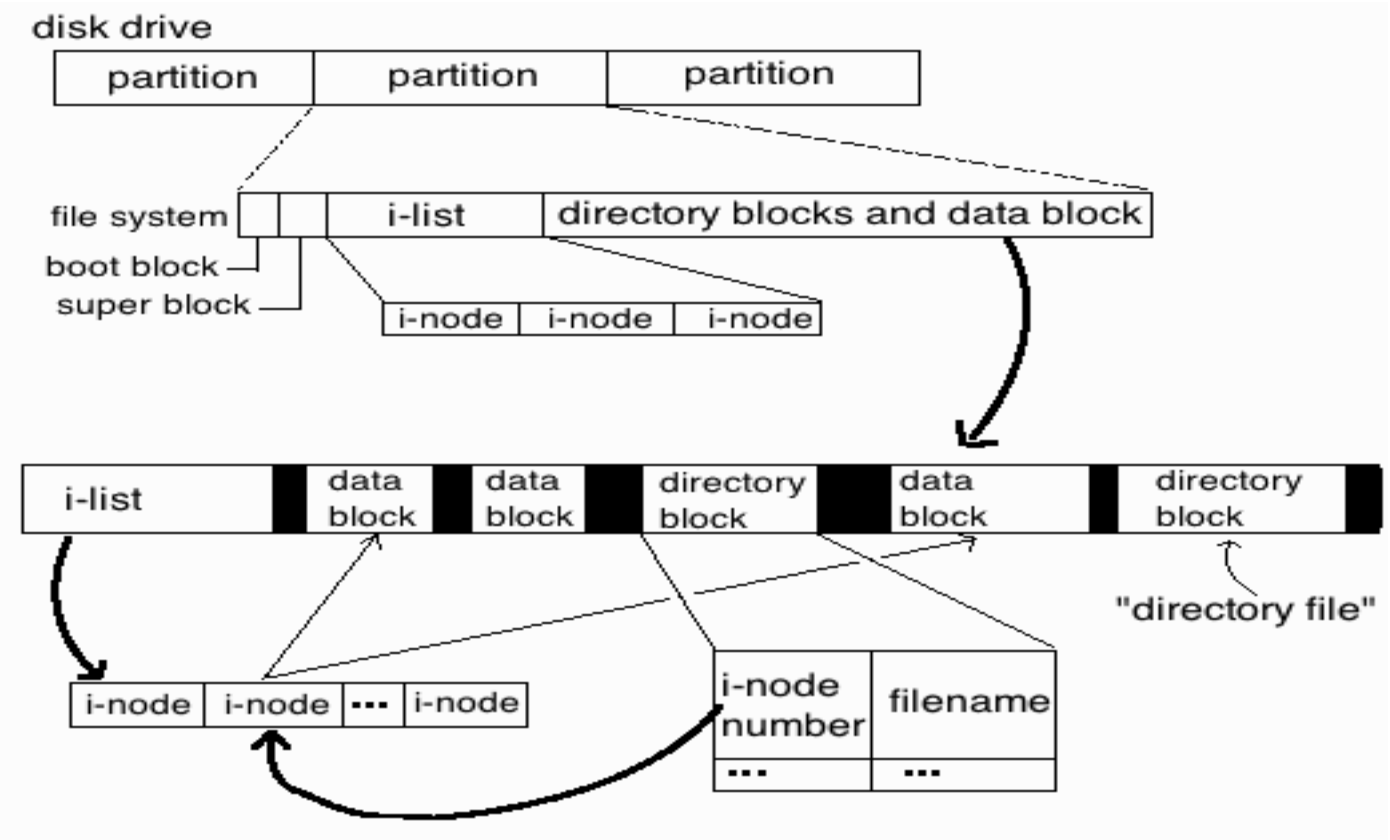


Different between Linux and Windows File System

Characteristic	Windows	Linux
File System	**NTFS, FAT**	**ext2,ext3**
Reference Root of each partition point	**Each partition will be own root** **Ex. C:, D:, F:**	**Each partitions is mounted under /**



Disk layout in classical Linux system


disk drive
partition
partition
partition
file system
i-list
directory blocks and data block
boot block
super block
i-node
i-node
i-node
i-list
data block
data block
directory block
data block
directory block
"directory file"
i-node
i-node
...
i-node
i-node number
filename
...
...

The System Programmer's Perspective

File System Calls

Return File Desc	Use of namei	Assign inodes	File Attributes	File I/O	File Sys Structure	Tree Manipulation
open creat dup pipe close	open creat chdir chroot chown chmod stat link unlink mknod mount umount	creat mknod link unlink	chown chmod stat	read write lseek	mount umount	chdir chown

Lower Level File System Algorithms

System Calls for File Management

System call	Description
fd = creat(name, mode)	One way to create a new file
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information
s = fstat(fd, &buf)	Get a file's status information
s = pipe(&fd[0])	Create a pipe
s = fcntl(fd, cmd, ...)	File locking and other operations

- **s** is an error code
- **fd** is a file descriptor
- **position** is a file offset

System Calls for Directory Management

System call	Description
s = mkdir(path, mode)	Create a new directory
s = rmdir(path)	Remove a directory
s = link(oldpath, newpath)	Create a link to an existing file
s = unlink(path)	Unlink a file
s = chdir(path)	Change the working directory
dir = opendir(path)	Open a directory for reading
s = closedir(dir)	Close a directory
dirent = readdir(dir)	Read one directory entry
rewinddir(dir)	Rewind a directory so it can be reread

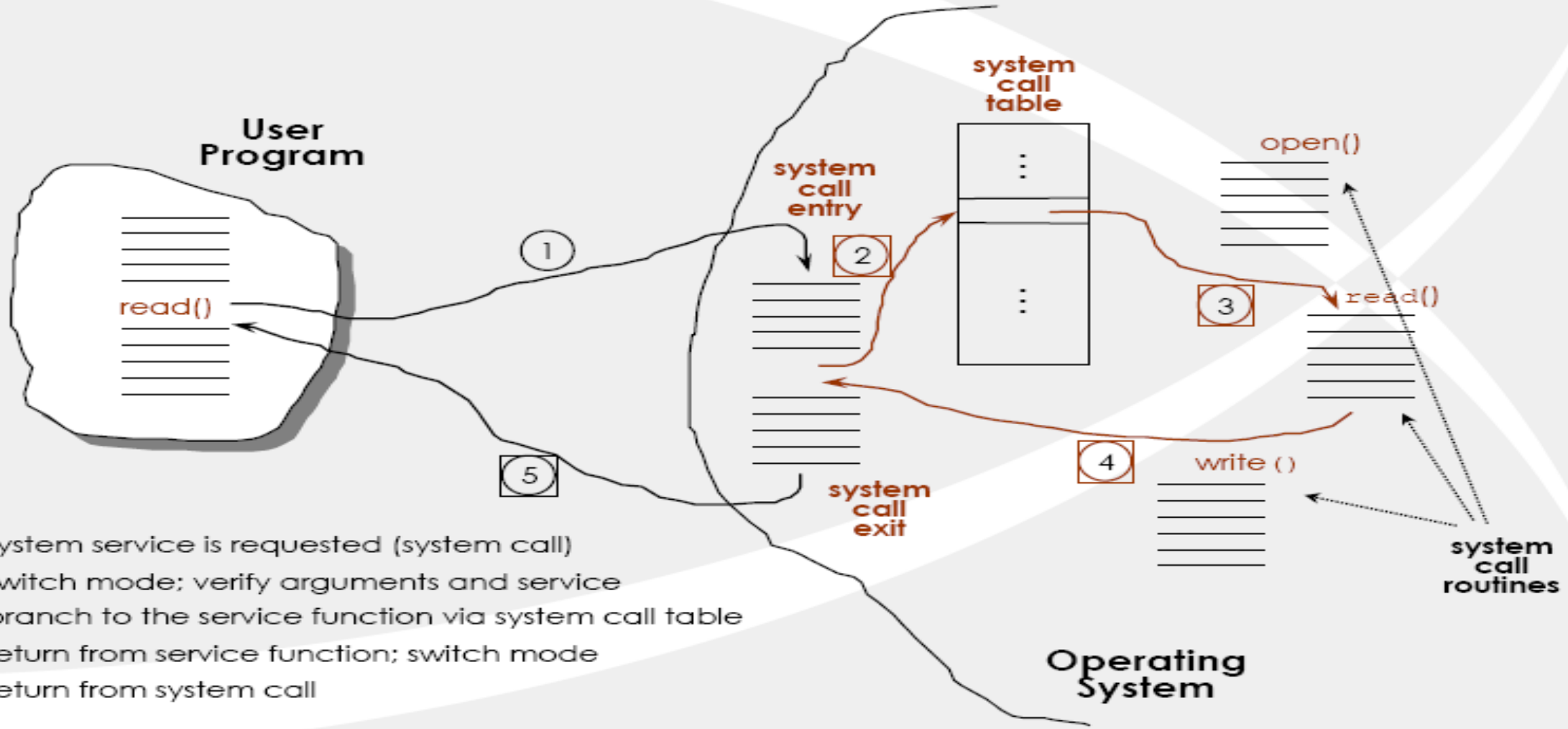
- **s** is an error code
- **dir** identifies a directory stream
- **dirent** is a directory entry

System Calls for File Protection

System call	Description
s = chmod(path, mode)	Change a file's protection mode
s = access(path, mode)	Check access using the real UID and GID
uid = getuid( )	Get the real UID
uid = geteuid( )	Get the effective UID
gid = getgid( )	Get the real GID
gid = getegid( )	Get the effective GID
s = chown(path, owner, group)	Change owner and group
s = setuid(uid)	Set the UID
s = setgid(gid)	Set the GID

- **s** is an error code
- **uid** and **gid** are the UID and GID, respectively

How system call is processed?

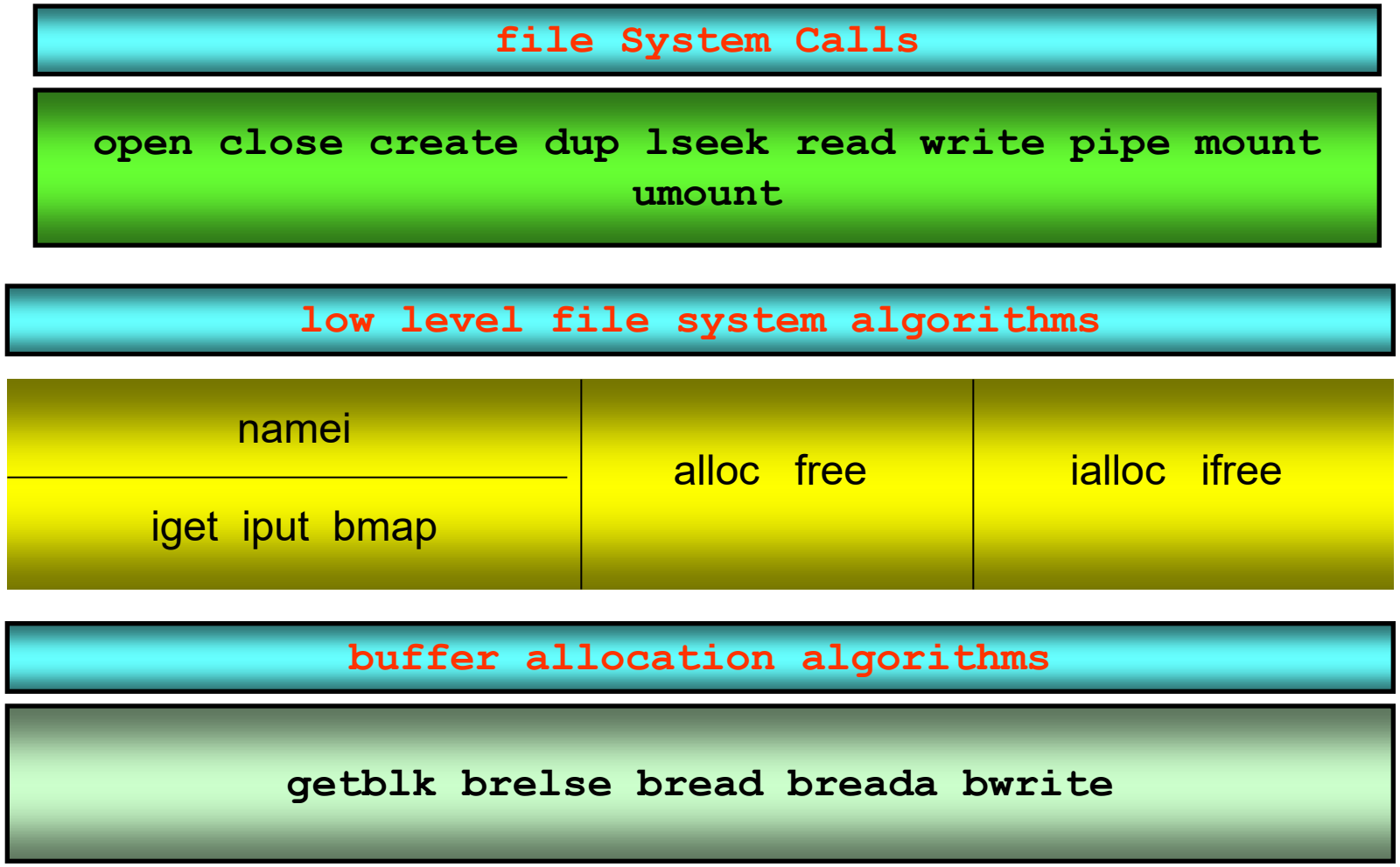


1. system service is requested (system call)
2. switch mode; verify arguments and service
3. branch to the service function via system call table
4. return from service function; switch mode
5. return from system call

file System Calls

open close create dup lseek read write pipe mount umount

File Manger Routines and Data Structures


file System Calls
open close create dup lseek read write pipe mount umount
low level file system algorithms
namei
iget iput bmap
alloc free
ialloc ifree
buffer allocation algorithms
getblk brelse bread breada bwrite

Inodes

Inode

- contains the information necessary for a process to access a file.

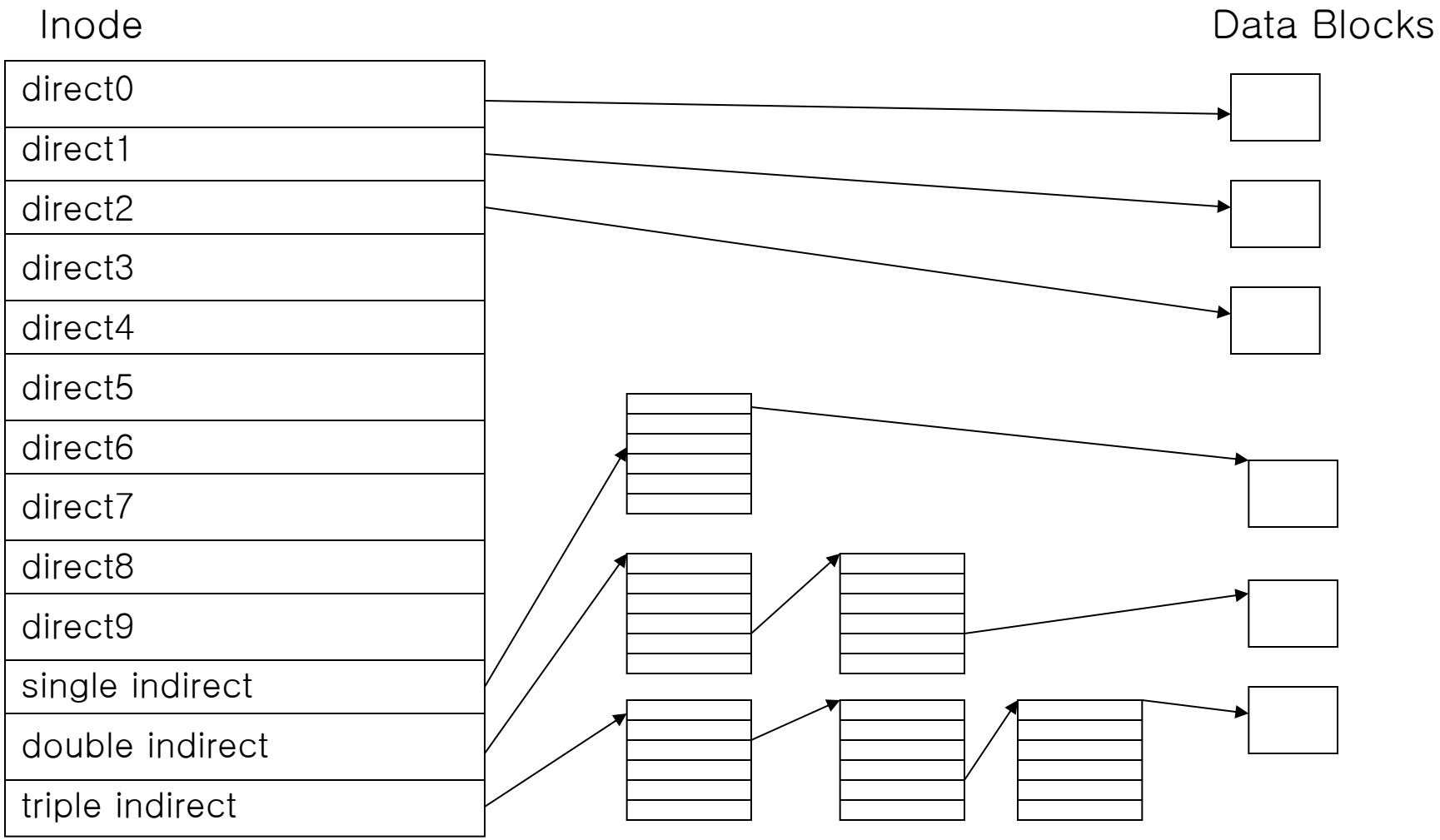
- exits in a static form on disk and the kernel (= file system manager) reads them into an in-core (= Main memory) inode.

המבנה (רשימת השדות) של רשומת inode

Disk inodes

- disk inode fields include:
  - The file's owner (a UID)
  - The file's group (a GID)
  - file type (e.g., file, directory, symbolic link)
  - The file's mode bits (also called file permissions or permission bits)
  - file access times
    - The time the file's inode was last modified (the ctime)
    - The time the file's contents were last modified (the mtime)
    - The time the file was last accessed (the atime) for read ( ), exec ( ), etc
  - A reference count: the number of names the file has (or: the number of links to the file)
  - The location of the file contents on the disk, if any
    - table with the address (= disk number) of the disk data blocks containing the contents of the file.
  - file size, in bytes, if applicable

Structure of a regular file



in-core inodes

- The in-core copy of the inode contains the following fields in addition to the fields of the disk inode:
  - status of the in-core inode
    - The inode is locked
    - A process is waiting for the inode to become unlocked
    - in-core copy differs from the disk copy as a result of a change to the data in the inode
    - in-core copy differs from the disk copy as a result of a change to the file data

in-core inodes

- logical device number of the file system that contains the file
- inode number
- pointers to other in-core inodes
  - The kernel links inodes on hash queues and on a free list in the same way that links buffers on a buffer hash queues and on the buffer free list.
  - A hash queue is identified according to the inode’s logical device number and inode number

in-core inodes

- reference count , indicating the number of instances of the file that are active (such as when *open*ed)

Directories

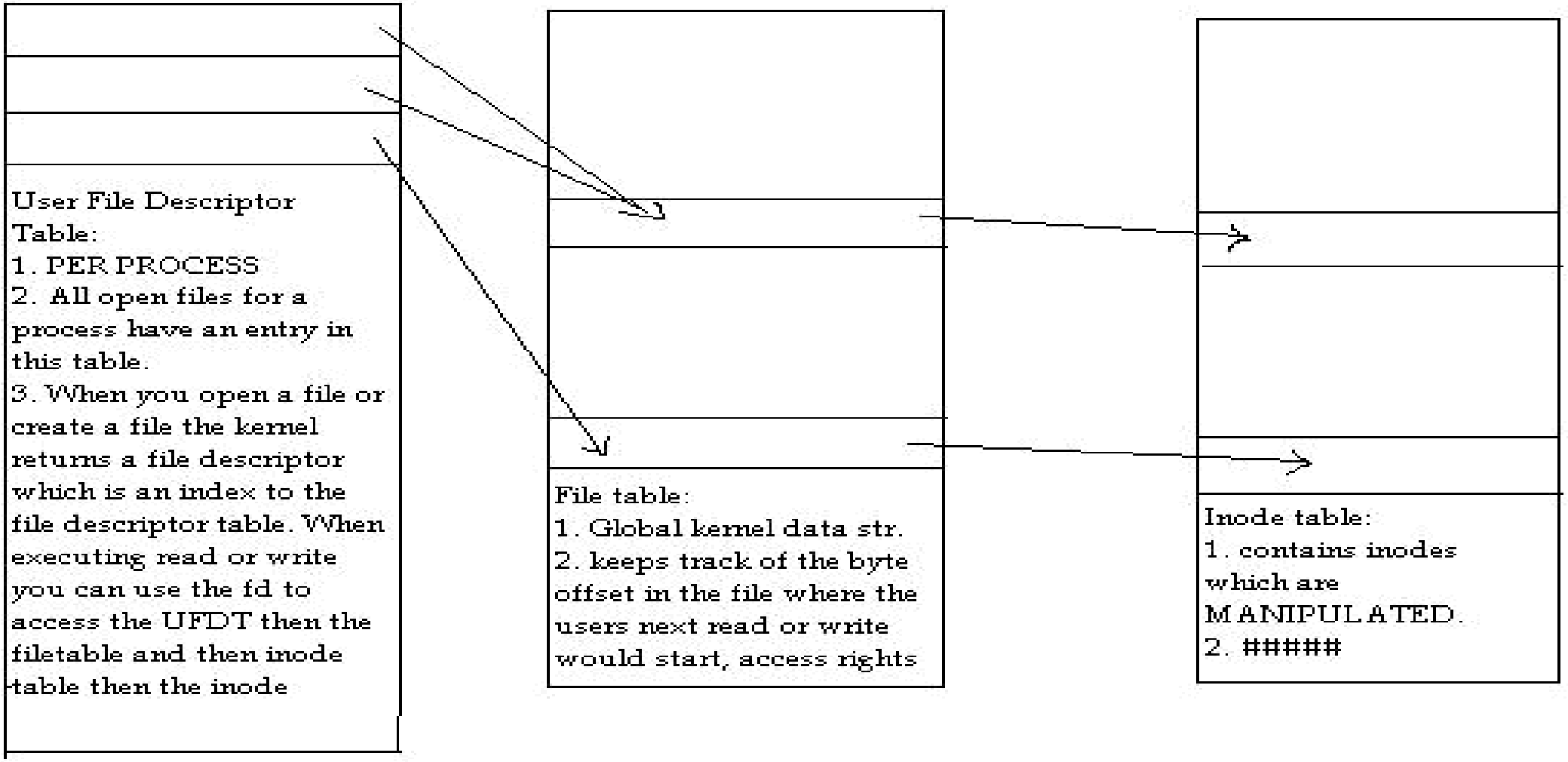
- Directories
  - A directory is a file whose data is a sequence of entries, each consisting of an inode number and the name of a file contained in the directory
  - Every directory contains the file name “.” and “..” whose inode numbers are those of the directory and its parent directory’ respectiveky.

- מבני הנתונים העיקריים המנוהלים ב- NVM:
  - טבלת ה- inodes
  - רשימת רשומות ה- inode הפנויות (והמוקצות)
  - רשימת בלוקי הנתונים הפנויים
- תחילתן של שתי הרשימות האחרונות לעיל שמורה בבלוק העל (= ה- superblock).

Disk (partition) layout in Linux Systems

- מבני הנתונים העיקריים המנוהלים הזיכרון הראשי:
  - in-core (= in memory) inode list
  - open file descriptor table
  - process file descriptor table

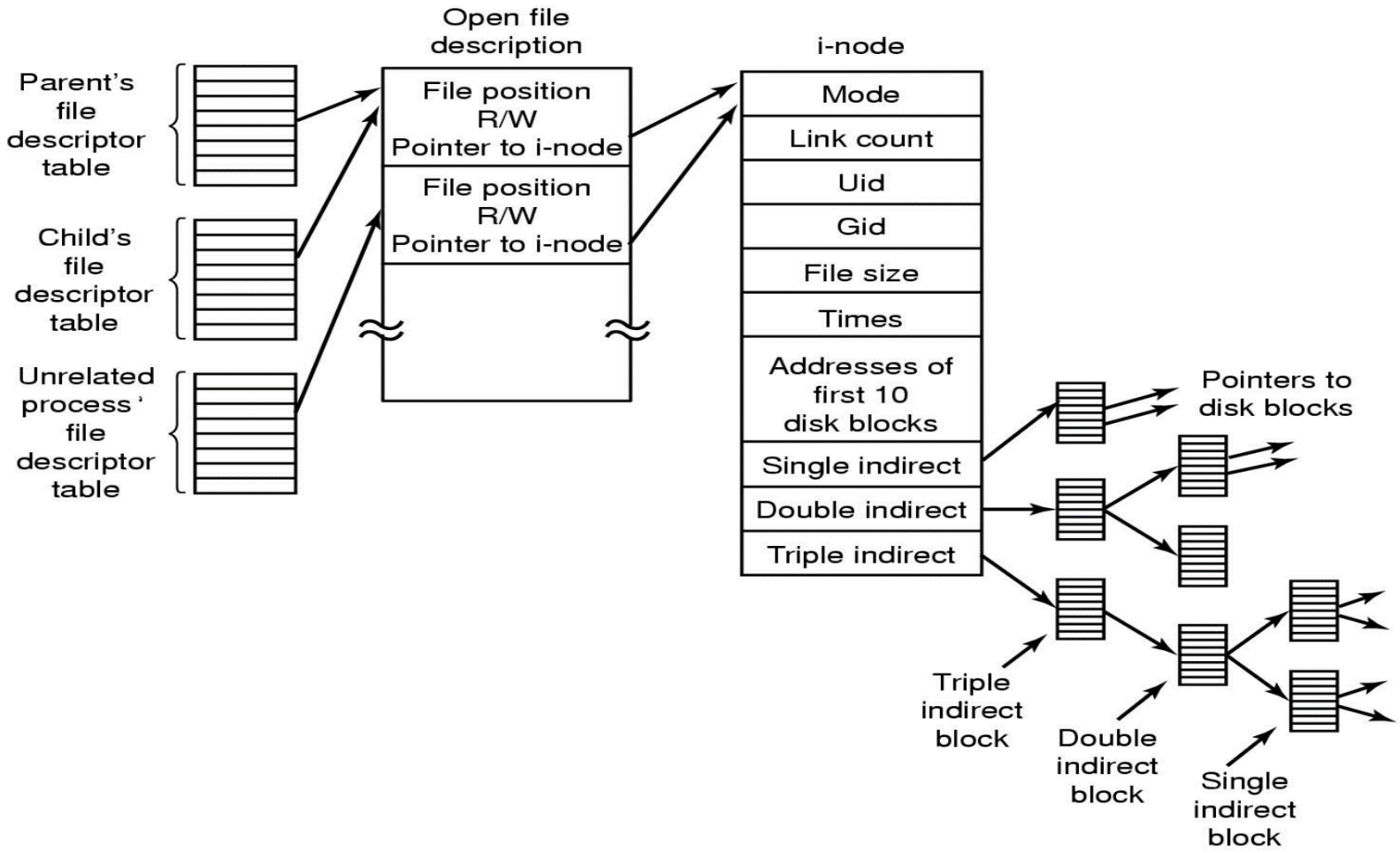
- שני מבני הנתונים הראשונים לעיל הם מבנים גלובליים, ומנוהלים עבור כלל התהליכים שרצים במערכת.
- מבנה הנתונים השלישי מנוהל לכל תהליך בנפרד (כחלק מרשומת ה- PCB שלו).



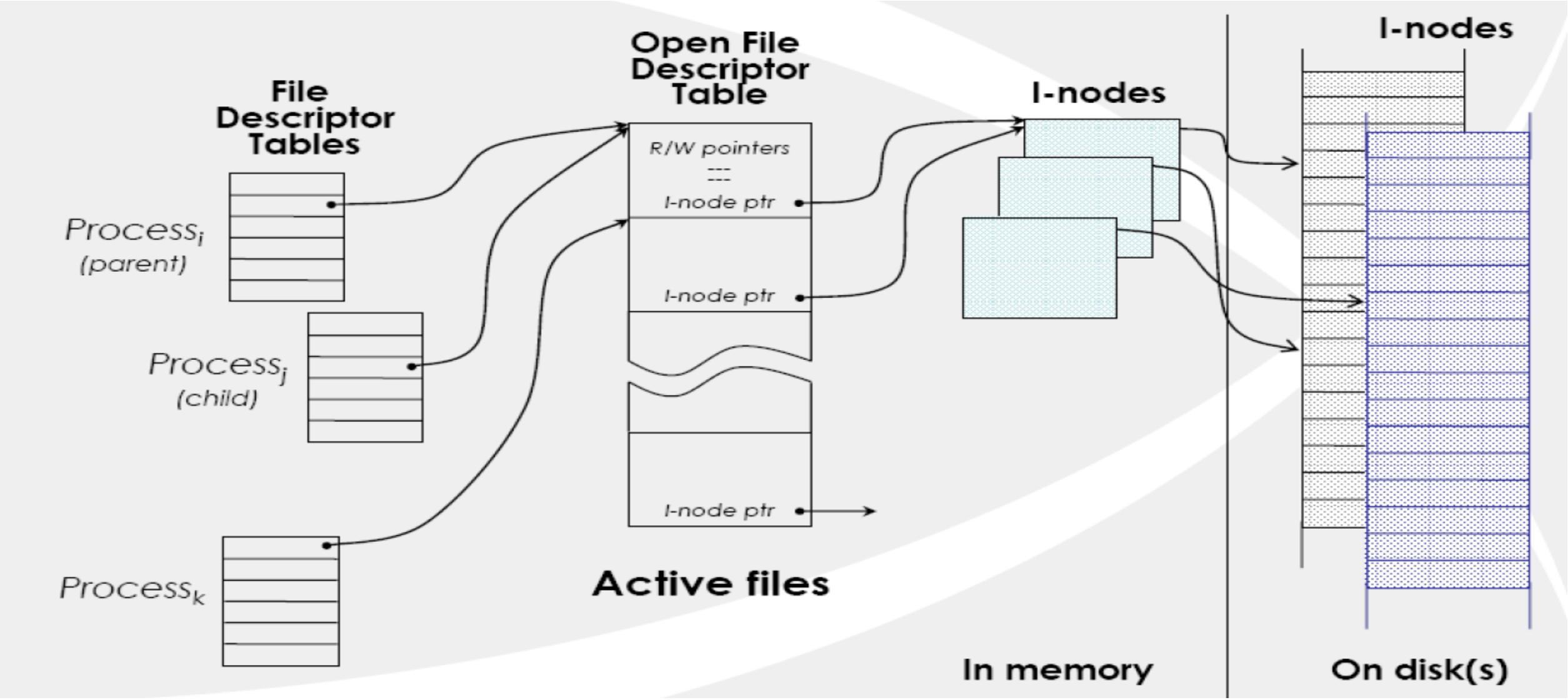
WHEN A PROCESS CREATES OR OPENS A FILE the kernel allocates an entry from each of the three tables. They are used to maintain the state of the file and the users access to it.

"Inodes are stored in the file system, but they are read in the Inode table when manipulating them.

Linux File System



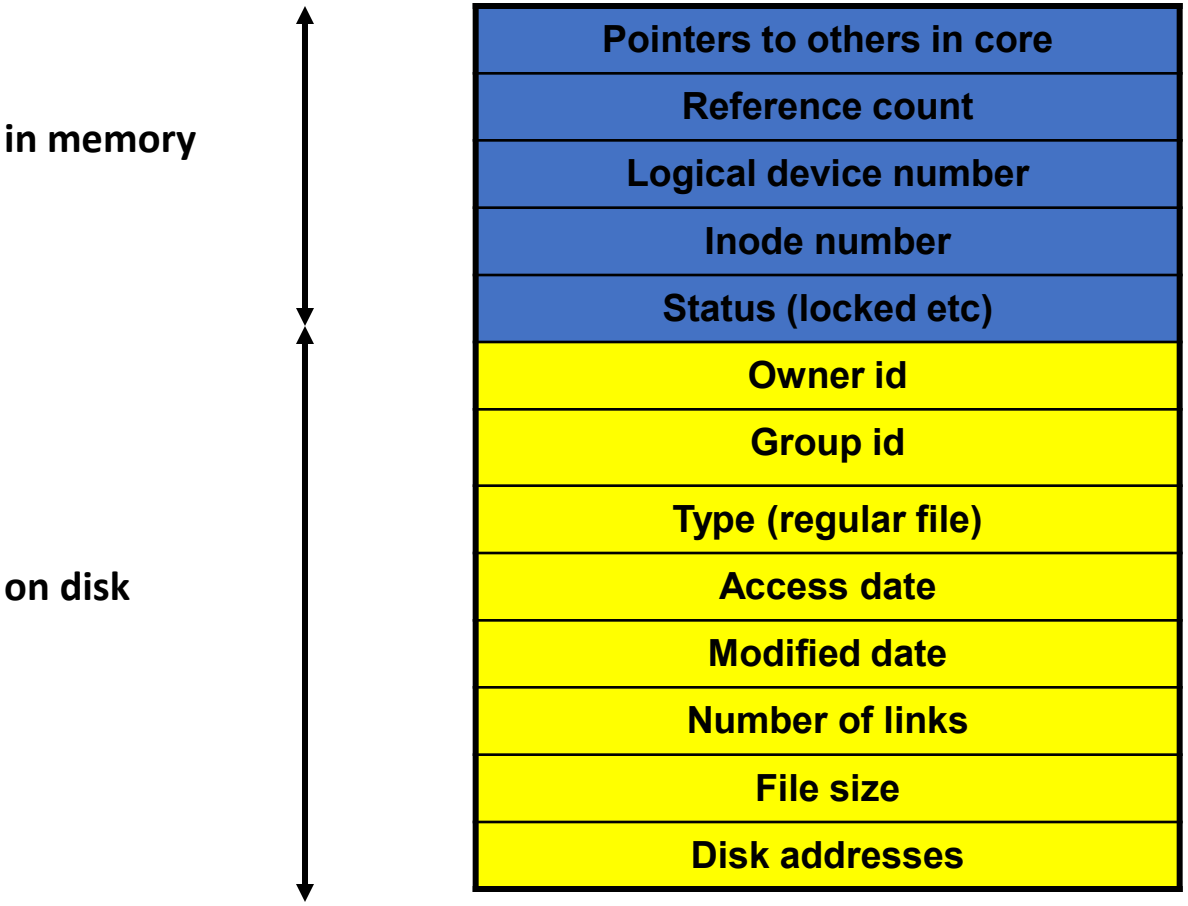
The relation between the file descriptor table, the open file description


File Descriptor Tables
Open File Descriptor Table
I-nodes
I-nodes
R/W pointers
I-node ptr
I-node ptr
I-node ptr
Process$_i$ (parent)
Process$_j$ (child)
Process$_k$
Active files
In memory
On disk(s)

in-core (= in memory) inode list

- רשימה זו כוללת את כל רשומות ה- inode של כל הקבצים הפתוחים במערכת המחשב.
- אם רשומת inode של קובץ F נמצאת ברגע נתון ברשימה הזו, אזי פירוש הדבר כי קיים ברגע זה במערכת המחשב <u>לפחות תהליך אחד</u> שפתח את הקובץ (וכמובן עדיין לא סגר אותו).
  - יתכן שתהליך מסוים P1 יפתח את הקובץ F מספר פעמים, מבלי לסגור אותו, ואז יש ברשימה זו עותק (יחיד) של רשומת ה- inode של F.
  - יתכן ששני תהליכים שונים P1 ו- P2 יפתחו את הקובץ F, פעם אחת או מספר פעמים מבלי לסגור אותו, ואז יש ברשימה זו עותק (יחיד) של רשומת ה- inode של F.

- לרשומת inode ששייכת ל- in-memory inode list יש אותם השדות כמו לרשומת inode ששייכת ל- inodes table שבדיסק הקשיח, אבל בנוסף לשדות אלו יש לה שדות נוספים.
- השקף הבא מציג את ההבדל בין מבנה רשומת inode שבדיסק לבין המבנה של רשומת inode שהיא in memory:
  - רשומת inode שבדיסק הקשיח מכילה את השדות הצבועים בצהוב בלבד.
  - רשומת inode שב- in memory מכילה הן את השדות הצבועים בצהוב והן את אלו הצבועים בכחול.


in memory
on disk
Pointers to others in core
Reference count
Logical device number
Inode number
Status (locked etc)
Owner id
Group id
Type (regular file)
Access date
Modified date
Number of links
File size
Disk addresses

open file descriptor table

- הטבלה הזו מכילה כניסה עבור כל פתיחה של קובץ.
- כל כניסה (= רשומה) בטבלה מכילה 4 שדות:
  - מצביע לרשומה ב- in-memory inode list
  - סמן קובץ
  - סוג פתיחת הקובץ (קריאה בלבד, כתיבה בלבד או קריאה וכתיבה)
  - מספר התהליכים המשתפים את סמן הקובץ

open()

- תהליך רוצה לגשת לקובץ (לקרוא ו/או לכתוב נתונים) חייב קודם לבקש ממנהל הקבצים לפתוח את הקובץ.
- קריאת המערכת לפתיחת קובץ היא open.

- התחביר (= syntax) של הקריאה ל- open הוא כדלקמן:

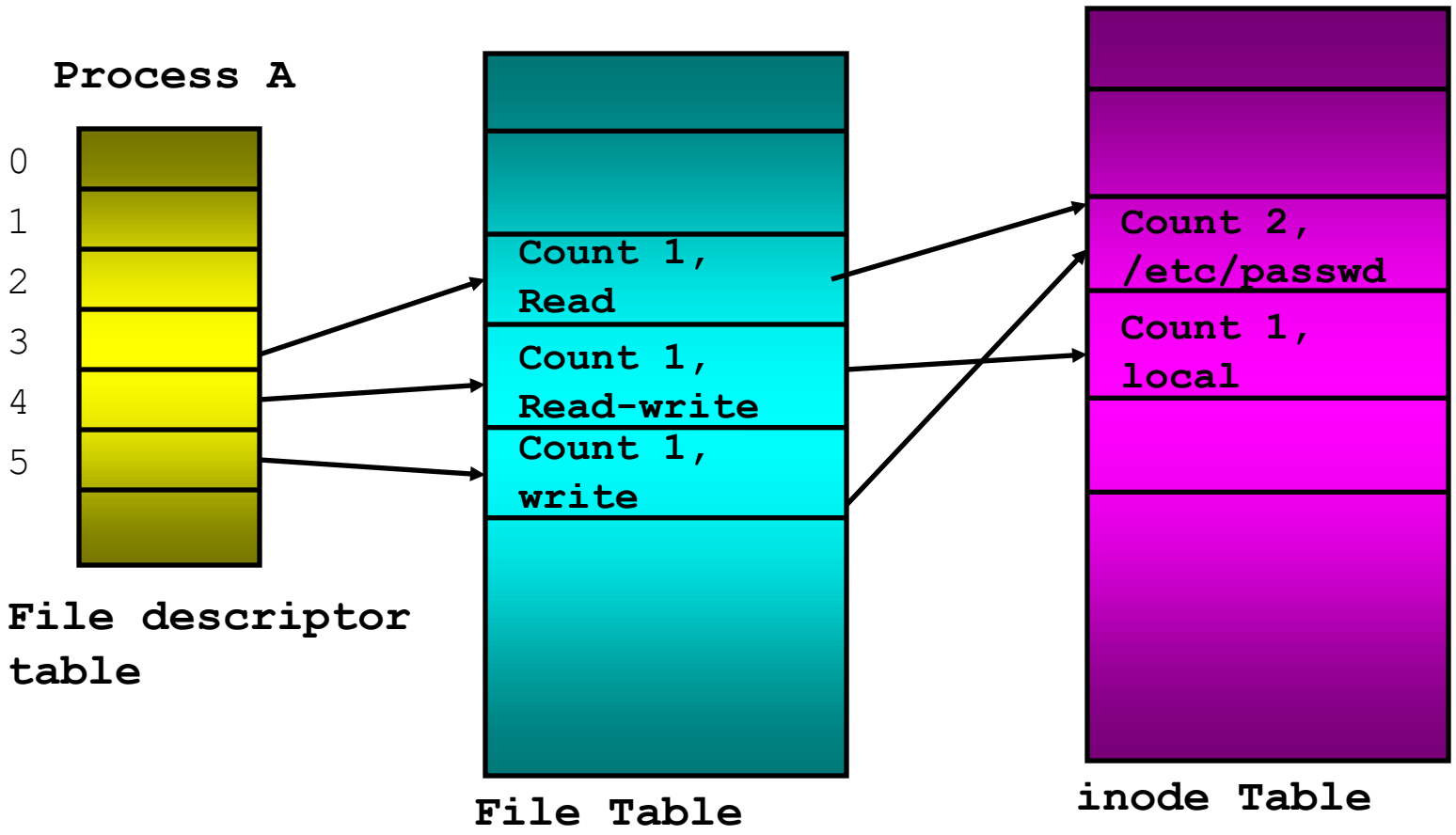
```
int open(const char *pathname, int flags);
```

- באשר:
- pathname = מחרוזת (= string) של תווים שמייצגת את שם הקובץ
  - full pathname או relative pathname
- flags = אופן פתיחת הקובץ
  - לדוגמא: O_RDONLY, O_WRONLY, O_RDWR, O_APPEND וכו'

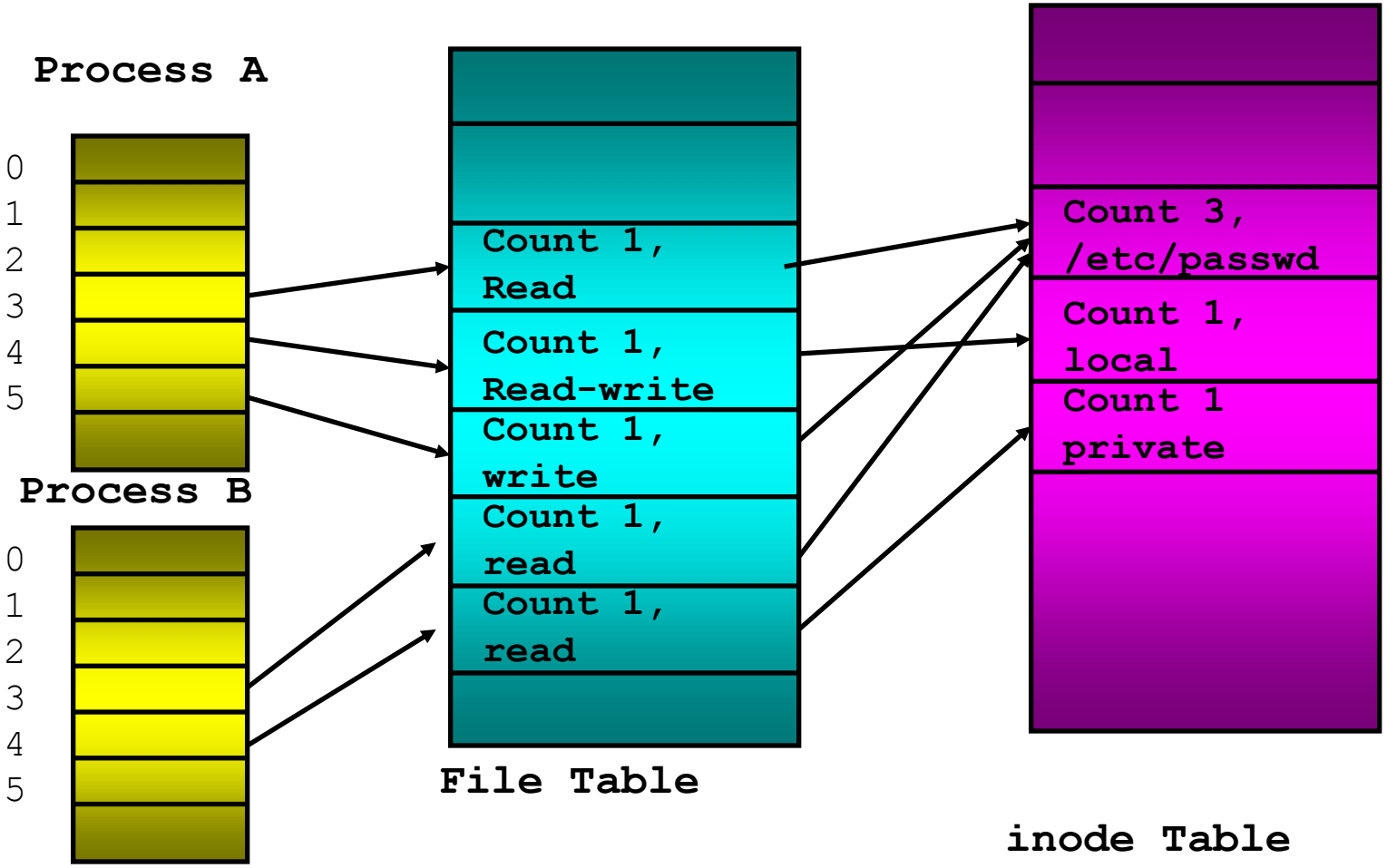
- הערך ש- open מחזירה הוא מספר שלם:
  - במקרה שפעולת פתיחת הקובץ הצליחה open מחזירה מספר שלם אי-שלילי (≤ 0).
  - במקרה שפעולת פתיחת הקובץ נכשלה open מחזירה 1-.

- הערך ש- open מחזירה נקרא file descriptor.

- ה- file descriptor הוא אינדקס של כניסה בטבלת הקבצים הפתוחים של התהליך שקרא ל- open.
- הכניסה תכיל מצביע לכניסה ב- open file descriptor table.
- הכניסה ב - open file descriptor table תכיל:
  - מצביע לרשומת inode ב- in-memory inode list
  - ערכו ההתחלתי של סמן הקובץ יהיה 0 אלא אם כן הקובץ נפתח עם O_APPEND
  - סוג פתיחת הקובץ (= O_RDONLY, O_WRONLY, O_RDWR
  - שדה count שערכו לצורך הדיון בקורס יהיה תמיד 1
    - הערה: באופן כללי ערכו של השדה count יכול להיות גדול מ- 1, אבל אנחנו לא נתאר ולא נדון במקרים כאלו בקורס הנוכחי.
    - הערה נוספת: לשים לב שלא לבלבל בין השדה count שברשומות (כניסות) של ה- open file descriptor table לבין שדה בשם כזה שנמצא ברשומות inode שב- in-memory inode list.




```
fd1= open ("/etc/ passwd", O_RDONLY);
fd2 = open ("local". RDWR);
fd3 = open ("/etc/ passwd ",O_WRONLY);
```



fd1=open ("/etc/ passwd", O_RDONLY);
fd2 = open ("private". O_RDONLY);

```
algorithm open
input: pathname
        type of open
output: file descriptor
{
        convert file name into inode (algorithm namei)
        if (file doesn't exist or not permitted access)
                return (error)
        allocate file table entry for inode, initialize count and offset;
        allocate user file descriptor entry, set pointer to file table entry;
        if (type of open specifies truncated file)
                free all file blocks (algorithm free)
        unlock inode;  /* locked above in namei */
        return (user file descriptor);
}
```

read()

- כמובן שתנאי הכרחי, אבל לא מספיק, לכך שהקריאה ל- read(...) תצליח הוא שהתהליך פתח בהצלחה את הקובץ ממנו הוא רוצה לקרוא מידע.
  - פתיחת הקובץ נעשית על ידי קריאת המערכת open(...)
  - הקריאה ל- open מחזירה ערך מספר שלם (= int) שנקרא file descriptor
  - ה- file descriptor שמחזירה open מועבר כארגומנט הראשון בקריאה ל- read.

- אם התהליך לא פתח את הקובץ קודם לקריאה ל- read, או אם הוא ניסה לפתוח את הקובץ וניסיון הפתיחה נכשל, אזי הקריאה ל- read תוביל לסיום מיידי של ריצת התהליך.

Sample Program for Reading a File

```
#include  <fcntl.h>
main()
{
        int fd;
        char lilbuf[20], bigbuf[1024];

        fd = open("/etc/passwd", O_RDONLY);
        read(fd, lilbuf, 20);
        read(fd, bigbuf, 1024);
        read(fd, lilbuf, 20);
}
```